Homework 08: Margins

Questions 1 and 2

1 Question 1: Boundary and margin

1.1 Given rule

The question gives the score

$$f(\mathbf{x}) = 0.8x_1 + 0.6x_2.$$

The coefficient vector has length 1, so the perpendicular distance from an observation to the decision boundary, $f(\mathbf{x}) = 0$, is $|f(\mathbf{x})|$. For observation i, the functional margin is $y_i f(\mathbf{x}_i)$. Comparing it with 1 determines whether the observation is inside, on, or beyond the margin for its class.

1.2 Calculations

The code applies the supplied score to each row; it does not fit a classifier.

1.3 Results by observation

The distance is always nonnegative. The functional margin includes the class label, so it retains a negative sign when an observation is misclassified. Being inside the margin does not by itself mean the observation is misclassified.

```
results <- data.frame(
  Observation = observations$observation,
  `Distance to boundary` = round(observations$distance, 2),
  `Functional margin` = round(observations$functional_margin, 2),
  `Margin position` = observations$margin_position,
  `Misclassified` = ifelse(observations$misclassified, "Yes", "No"),
  check.names = FALSE
)

kable(
  results,
  caption = "Distance and functional margin for each observation."
)
```

Table 1: Distance and functional margin for each observation.

Observation	Distance to boundary	Functional margin	Margin position	Misclassified
1	1.00	1.00	On	No
2	1.35	1.35	Beyond	No
3	1.50	1.50	Beyond	No
4	1.70	1.70	Beyond	No
5	1.90	1.90	Beyond	No
6	2.20	2.20	Beyond	No
7	2.40	2.40	Beyond	No
8	0.64	0.64	Inside	No
9	0.20	0.20	Inside	No
10	0.20	-0.20	Inside	Yes
11	1.00	1.00	On	No
12	1.30	1.30	Beyond	No
13	1.55	1.55	Beyond	No
14	1.75	1.75	Beyond	No
15	1.95	1.95	Beyond	No
16	2.15	2.15	Beyond	No
17	2.45	2.45	Beyond	No
18	0.64	0.64	Inside	No
19	0.24	0.24	Inside	No
20	0.20	-0.20	Inside	Yes

1.4 Plot of the boundary and margins

Points are colored by class. The solid line is the decision boundary, and the dashed lines are the margins. Black rings and labels identify the observations inside their class margin.

```
inside_observations <- observations[
  observations$margin_position == "Inside",
]

# Manually position labels to keep nearby observation numbers distinct.
label_positions <- data.frame(
  observation = c(8, 9, 10, 18, 19, 20),
  label_x = c(-0.72, 0.02, 0.72, -0.78, -0.12, 0.70),
  label_y = c(0.52, 0.56, 0.66, -0.72, -0.82, -0.48)
)

inside_observations <- merge(
  inside_observations,
  label_positions,
  by = "observation",
  sort = FALSE
)

ggplot(observations, aes(x = x1, y = x2)) +
  geom_abline(
    aes(
      intercept = 0,
      slope = -0.8 / 0.6,
```

```
    linetype = "Decision boundary"
  ),
  color = "black",
  linewidth = 0.9
) +
geom_abline(
  aes(
    intercept = 1 / 0.6,
    slope = -0.8 / 0.6,
    linetype = "Margin lines"
  ),
  color = "gray35",
  linewidth = 0.7
) +
geom_abline(
  aes(
    intercept = -1 / 0.6,
    slope = -0.8 / 0.6,
    linetype = "Margin lines"
  ),
  color = "gray35",
  linewidth = 0.7
) +
geom_point(aes(color = factor(y)), size = 3) +
geom_point(
  data = inside_observations,
  shape = 1,
  color = "black",
  size = 5,
  stroke = 1
) +
geom_text(
  data = inside_observations,
  aes(x = label_x, y = label_y, label = observation),
  color = "black",
  size = 3.4,
  show.legend = FALSE
) +
scale_color_manual(
  name = "Class y",
  values = c("-1" = "#D55E00", "1" = "#0072B2"),
  labels = c("-1", "1")
) +
scale_linetype_manual(
  name = "Reference lines",
  values = c("Decision boundary" = "solid", "Margin lines" = "dashed")
) +
scale_x_continuous(
  name = expression(x[1]),
  limits = c(-3.2, 3.2),
  breaks = seq(-3, 3, by = 1),
  expand = c(0, 0)
) +
```

```r
  scale_y_continuous(
    name = expression(x[2]),
    limits = c(-3, 3),
    breaks = seq(-3, 3, by = 1),
    expand = c(0, 0)
  ) +
  coord_fixed(ratio = 1) +
  labs(
    title = "Boundary and margin for the supplied score",
    subtitle = "Rings and labels mark observations inside their class margin",
    caption = "The given rule is used; no classifier is fitted."
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")
```



Figure 1: Observations and the fixed decision boundary with its two margin lines.

1.5 Margin counts and misclassifications

Count the observations in each margin category. Misclassification is checked separately by comparing the predicted class with the observed class.

```
margin_counts <- table(
  factor(
    observations$margin_position,
    levels = c("Inside", "On", "Beyond")
  )
)

counts <- data.frame(
  `Margin position` = names(margin_counts),
  Count = as.integer(margin_counts),
  check.names = FALSE
)

kable(counts, caption = "Number of observations in each margin position.")
```

Table 2: Number of observations in each margin position.

Margin position	Count
Inside	6
On	2
Beyond	12

```
misclassified_observations <- observations$observation[
  observations$misclassified
]
misclassified_observations
```

```
## [1] 10 20
```

There are **6 observations inside**, **2 on**, and **12 beyond** the margin. Observations **10 and 20** are misclassified. Both are inside their class margin; their negative functional margins identify the classification errors.

2 Question 2: Margin in four dimensions

2.1 Projection and geometric margin

The supplied score is

$$f(\mathbf{x}) = -1 + \mathbf{x}^\top \begin{pmatrix} 1 \\ 2 \\ -2 \\ 4 \end{pmatrix}.$$

The coefficient vector is perpendicular to the decision boundary. To show the observations on a one-dimensional axis, project each point onto the **unit** direction of that vector. The boundary's coordinate on this axis is $-\beta_0/\|\beta\|$. The geometric distance from an observation to the boundary is its absolute score divided by $\|\beta\|$.

The axis shows each observation's projection onto the unit coefficient direction. The vertical dashed line marks the decision boundary. The red ring identifies the observation(s) with the smallest distance to it.

```
observations_4d$class <- factor(
  observations_4d$y,
  levels = c(-1, 1),
  labels = c("y = -1", "y = +1")
)
closest_observations$class <- factor(
  closest_observations$y,
  levels = c(-1, 1),
  labels = c("y = -1", "y = +1")
)

ggplot(
  observations_4d,
  aes(x = projection, y = class, color = class)
) +
  geom_vline(
    xintercept = boundary_projection,
    linetype = "dashed",
    color = "black",
    linewidth = 0.8
  ) +
  geom_segment(
    data = closest_observations,
    aes(
      x = boundary_projection,
      xend = projection,
      y = class,
      yend = class
    ),
    inherit.aes = FALSE,
    color = "red",
    linewidth = 0.7,
    linetype = "dotted"
  ) +
  geom_point(
    size = 2.8,
    alpha = 0.8
  ) +
  geom_point(
    data = closest_observations,
    shape = 1,
    size = 5,
    stroke = 1.2,
    color = "red"
  ) +
  scale_color_manual(
```

```
    name = "Observed class",
    values = c("y = -1" = "#D55E00", "y = +1" = "#0072B2")
  ) +
  labs(
    x = "Projection onto unit coefficient direction",
    y = NULL,
    title = "Four-dimensional observations projected onto one axis",
    subtitle = paste(
      "Dashed line: decision boundary;",
      "red ring: closest observation(s)"
    )
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")
```

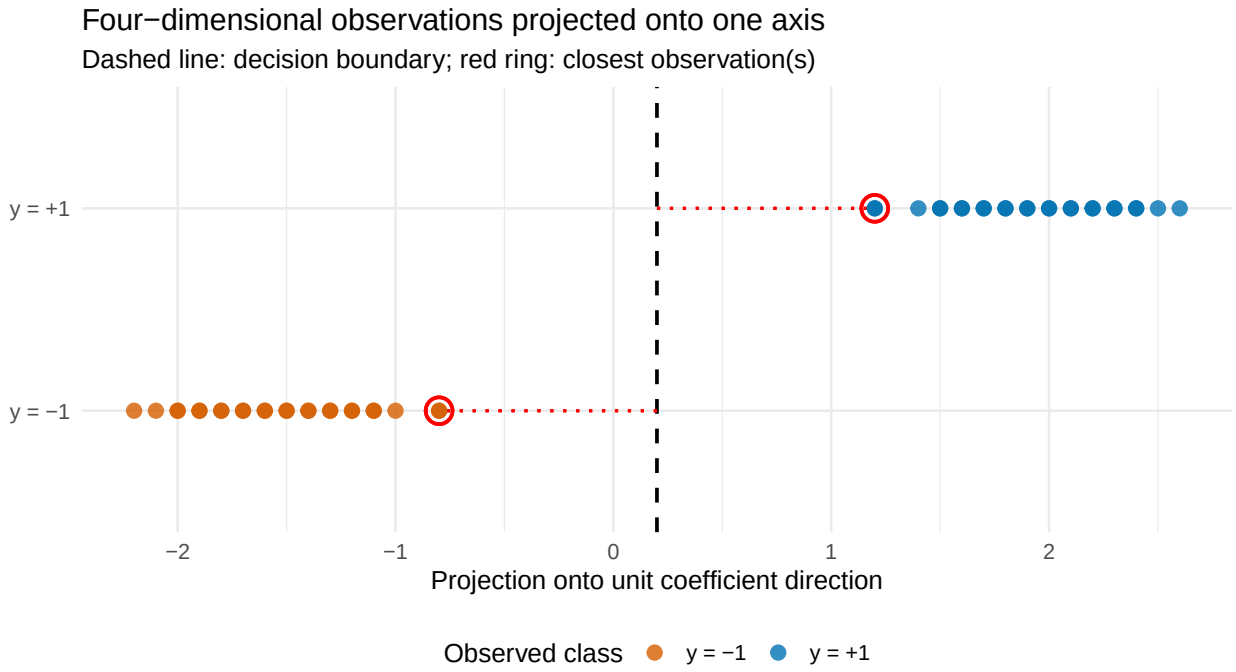


Figure 2: Observations projected onto the unit direction of the coefficient vector.

The geometric margin is the distance from the boundary to the closest observation. The minimum functional margin is also reported because it is needed to put the same boundary in SVM normalization.

```
margin_results <- data.frame(
  Quantity = c(
    "Norm of coefficient vector",
    "Boundary coordinate on projected axis",
    "Minimum functional margin",
    "Geometric margin",
    "Closest observation number(s)"
  ),
  Value = c(
    sprintf("%.4f", beta_norm),
```

```
    sprintf("%.4f", boundary_projection),
    sprintf("%.4f", minimum_functional_margin),
    sprintf("%.4f", minimum_geometric_distance),
    paste(closest_observations$observation, collapse = ", ")
  )
)

kable(
  margin_results,
  col.names = c("Quantity", "Value"),
  caption = "Projection and margin calculations."
)
```

Table 3: Projection and margin calculations.

Quantity	Value
Norm of coefficient vector	5.0000
Boundary coordinate on projected axis	0.2000
Minimum functional margin	5.0000
Geometric margin	1.0000
Closest observation number(s)	4, 15, 33, 36

2.2 SVM normalization

Multiplying the score by a positive constant preserves the zero-score decision boundary. To set the smallest functional margin to 1, divide both the original coefficient vector and intercept by the original minimum functional margin.

```
svm_parameters <- data.frame(
  Parameter = c("Intercept", names(beta)),
  `SVM-normalized value` = sprintf(
    "%.4f",
    c(beta_0_svm, beta_svm)
  ),
  check.names = FALSE
)

kable(
  svm_parameters,
  caption = "Intercept and coefficients under SVM normalization."
)
```

Table 4: Intercept and coefficients under SVM normalization.

Parameter	SVM-normalized value
Intercept	-0.2000
x1	0.2000
x2	0.4000
x3	-0.4000
x4	0.8000

```r
normalized_minimum_margin <-
  min(observations_4d$y * (
    beta_0_svm + as.vector(x_matrix %*% beta_svm)
  ))
```

The normalized minimum functional margin is 1.0000, confirming the requested normalization. Since the original functional margins are positive, this rescaling keeps their signs and leaves the decision boundary unchanged.